

SECUREVAULT

LAPORAN PROYEK AKHIR KRIPTOGRAFI

Dosen Pengampu: Lia Farhatuaini, S.Kom., M.Cs.



Disusun oleh:

Faiz Alfaresi (2488010012)

Mohammad Sofyan Nurseha (2488010031)

Faiz Muzaki Irsyad (2488010047)

Mohammad Satria Dinilhaq (2488010064)

JURUSAN INFORMATIKA

FAKULTAS ILMU TARBIYAH DAN KEGURUAN

UIN SIBER SYEKH NURJATI CIREBON

2026

A. PENDAHULUAN

1.1 Latar Belakang

Layanan penyimpanan cloud seperti Google Drive dan Dropbox memang memudahkan akses dan kolaborasi data. Namun, kelemahan utamanya adalah server dapat membaca data pengguna karena file disimpan dalam bentuk asli (plaintext). Selain itu, server juga memiliki akses teknis untuk membaca konten file, yang dapat dimanfaatkan oleh pihak ketiga atas nama kepatuhan hukum.

Fitur berbagi file pada layanan konvensional juga tidak dilindungi kriptografi yang memadai. Server bertindak sebagai perantara dan masih bisa mengakses isi file yang dibagikan antar pengguna, karena proses dekripsi terjadi di server sebelum file dikirim ke penerima. Hal ini sangat tidak ideal untuk dokumen sensitif seperti kontrak bisnis atau data pribadi.

SecureVault hadir untuk menjawab masalah tersebut dengan menerapkan *end-to-end encryption* pada sistem manajemen file. Data dienkripsi sebelum dikirim ke server, sehingga server hanya menyimpan *ciphertext* yang tidak bermakna. Fitur berbagi file juga dilindungi dengan mekanisme *key wrapping*, memastikan hanya penerima yang dituju yang dapat mendekripsi file.

1.2 Tujuan Sistem

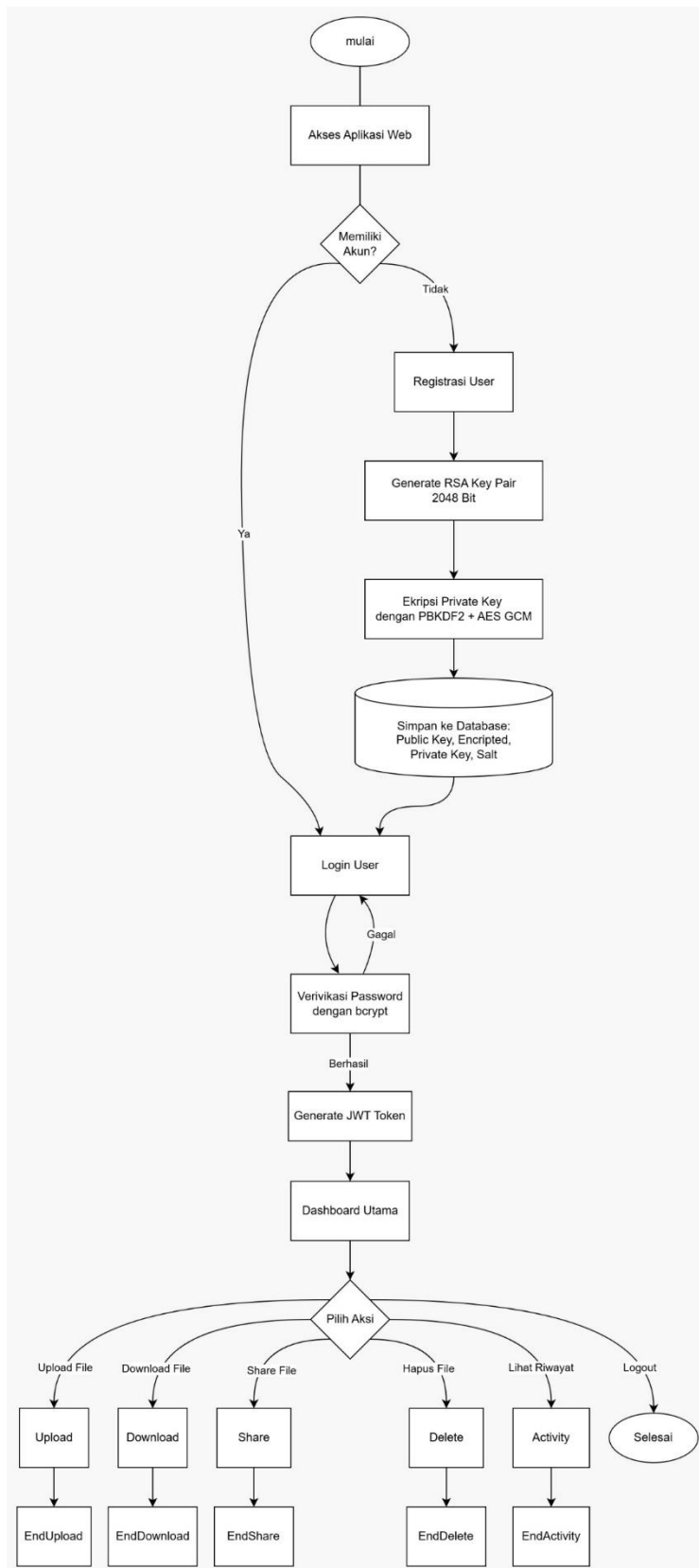
1. Kerahasiaan: File dienkripsi dengan session key acak sehingga server tidak bisa membaca isi file.
2. Integritas: AES-256-GCM + SHA-256 untuk memastikan file tidak dimodifikasi.
3. Berbagi dengan aman: Key wrapping memungkinkan berbagi file tanpa server terlibat dalam dekripsi.
4. Kontrol Pengguna: Pemilik file dapat menghapus atau mencabut akses kapan saja.
5. Otentikasi yang aman: Password user tidak pernah disimpan dalam bentuk plaintext.
6. Hybrid Cryptosystem: Menggabungkan kecepatan AES untuk enkripsi file besar dengan keamanan RSA untuk manajemen kunci dalam fitur berbagi file.

1.3 Pengguna yang Dituju

1. Individu: Menyimpan dokumen pribadi secara aman di cloud.
2. Tim/Organisasi Kecil: Kolaborasi aman dengan fitur berbagi file dan kontrol akses.

B. ARSITEKTUR SISTEM

2.1 Flowchart Sistem Secara Umum



2.2 Komponen-Komponen Sistem

2.2.1 Frontend

Komponen	Teknologi	Fungsi
User Interface	React + TypeScript	Menampilkan halaman login, register, dashboard
State Management	React Hooks (useState, useEffect)	Mengelola state aplikasi (user, file list, modal)
HTTP Client	Axios	Mengirim request ke backend, mengelola interceptor token
Drag & Drop	react-dropzone	Memudahkan user upload file
Styling	Tailwind CSS	Memberikan tampilan modern dan responsif

2.2.2 Backend

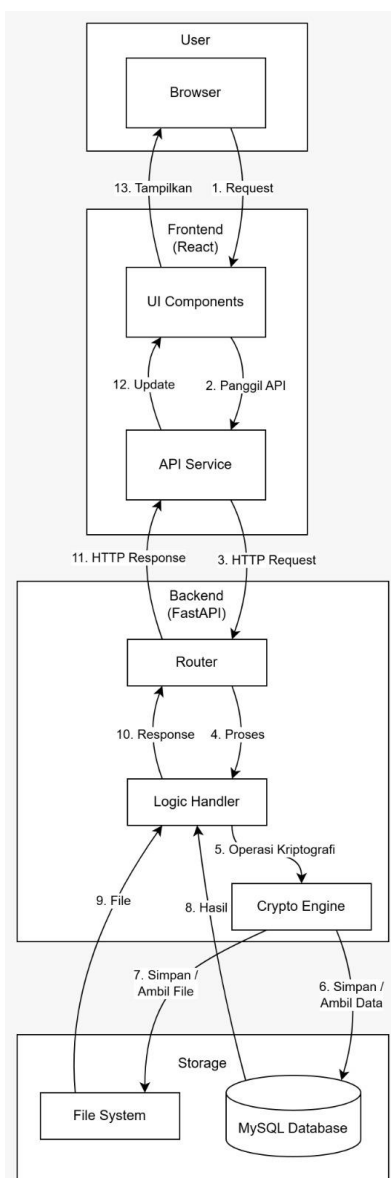
Komponen	Teknologi	Fungsi
Web Framework	FastAPI	Menangani routing, request/response, auto-documentation
Autentikasi	JWT + bcrypt	Verifikasi user, mengelola session
Database ORM	SQLAlchemy	Memetakan objek Python ke tabel MySQL
Validasi Data	Pydantic	Validasi request body dan response
CORS	FastAPI Middleware	Mengizinkan frontend mengakses API

2.2.3 Modul Kriptografi

Modul	Library	Fungsi
-------	---------	--------

RSA-2048-OAEP	cryptography	Wrapping/unwrapping session key
AES-256-GCM	cryptography	Enkripsi/dekripsi file
PBKDF2	cryptography	Turunkan kunci dari password
SHA-256	hashlib	Hashing untuk verifikasi integritas
bcrypt	bcrypt	Hashing password untuk login

2.3 Alur Data dari User ke Database



C. DESAIN KRIPTOGRAFI

3.1 Algoritma yang Dipilih

3.1.1 AES-256-GCM

Spesifikasi Teknis:

Parameter	Nilai
Algoritma	AES (Rijndael)
Panjang Kunci	256 bit
Mode Operasi	GCM (Galois/Counter Mode)
Panjang Nonce/IV	12 byte (96 bit)
Panjang Tag Autentikasi	16 byte (128 bit)

Alasan Pemilihan AES-256-GCM:

1. AES-256 belum pernah dipecahkan secara praktis dan memenuhi standar keamanan pemerintah AS untuk data *top secret*.
2. Mode GCM memberikan autentikasi terintegrasi, tidak seperti CBC atau ECB. Tag autentikasi dapat mendeteksi jika ciphertext telah dimodifikasi, sehingga integritas file terjamin.
3. Performa sangat cepat karena didukung instruksi hardware AES-NI pada prosesor modern.
4. Nonce unik per file (12 byte random) memastikan keamanan bahkan jika session key yang sama digunakan kembali.

Alternatif yang dipertimbangkan:

1. AES-256-CBC, ditolak karena tidak menyediakan autentikasi terintegrasi. Memerlukan HMAC terpisah yang menambah kompleksitas dan rawan terhadap *padding oracle attack*.
2. ChaCha20-Poly1305, sebenarnya alternatif yang bagus, namun AES lebih umum didukung oleh library dan hardware.

3.1.2 RSA-2048-OAEP

Spesifikasi Teknis

Parameter	Nilai
Algoritma	RSA

Panjang Kunci	2048 bit
Padding Scheme	OAEP (Optimal Asymmetric Encryption Padding)
Fungsi Hash untuk MGF1	SHA-256
Label	None (empty string)

Alasan Pemilihan RSA-2048-OAEP:

1. RSA adalah algoritma asimetris paling matang dan banyak digunakan untuk *hybrid encryption*.
2. OAEP (Optimal Asymmetric Encryption Padding) terbukti aman secara matematis terhadap *chosen-ciphertext attack*, berbeda dengan PKCS#1 v1.5 yang memiliki kelemahan historis (Bleichenbacher attack).
3. Memungkinkan *key wrapping* yang menjadi inti fitur berbagi file. Session key yang sama dapat dibungkus ulang dengan public key penerima yang berbeda.
4. Ukuran kunci 2048-bit direkomendasikan NIST aman hingga tahun 2030.

Alternatif yang Dipertimbangkan:

1. RSA-2048-PKCS#1 v1.5, ditolak karena padding lama yang rentan terhadap Bleichenbacher attack.
2. ECC/ECDH (Curve25519), ditolak karena implementasi key wrapping lebih kompleks dan membutuhkan library tambahan.

3.1.3 PBKDF2-HMAC-SHA256

Spesifikasi Teknis

Parameter	Nilai
Algoritma	PBKDF2
Pseudo-random function	HMAC-SHA256
Panjang output	32 byte (256 bit)
Iterasi	100.000
Salt	16 byte (128 bit) acak per user

Alasan Pemilihan PBKDF2-HMAC-SHA256:

1. Melindungi private key RSA yang disimpan di server. Private key dienkripsi dengan kunci yang diturunkan dari password user.
2. Tahan terhadap *brute force*. Dengan 100.000 iterasi, waktu komputasi untuk satu tebakan password menjadi lambat bagi attacker namun tidak terasa bagi user.
3. Salt unik per user (16 byte random) mencegah serangan *rainbow table*. Dua user dengan password yang sama akan menghasilkan kunci enkripsi berbeda.
4. Output panjangnya dapat dikontrol tepat 32 byte, sesuai kebutuhan kunci AES-256.

Alternatif yang Dipertimbangkan:

1. Argon2id, sebenarnya lebih baik (pemenang Password Hashing Competition 2015), namun membutuhkan dependency tambahan sementara PBKDF2 sudah tersedia di library cryptography
2. Scrypt, ditolak karena memerlukan banyak memori, bisa terlalu berat untuk environment dengan resource terbatas

3.1.4 bcrypt

Spesifikasi Teknis:

Parameter	Nilai
Algoritma	bcrypt (berbasis Blowfish)
Cost Factor (Rounds)	12 ($2^{12} = 4096$ iterasi)
Panjang Salt	128 bit (terintegrasi dalam output)
Panjang Output Hash	60 karakter (Base64 encoded)

Alasan pemilihan bcrypt:

1. Digunakan untuk hashing password saat registrasi dan verifikasi saat login.
2. Tahan terhadap *brute force* dengan cost factor yang dapat disesuaikan.
3. Salt terintegrasi otomatis dalam output hash, tidak perlu menyimpan salt terpisah seperti PBKDF2.
4. Merupakan standar industri untuk hashing password, digunakan oleh Django, Laravel, dan Ruby on Rails.

Alternatif yang Dipertimbangkan:

1. SHA256 langsung, ditolak karena terlalu cepat untuk hashing password.

D. IMPLEMENTASI

4.1 Implementasi Kriptografi

4.1.1 RSA Key Pair

Potongan Kode:

```
def generate_rsa_keypair():  
    """Generate RSA-2048 key pair"""  
    private_key = rsa.generate_private_key(  
        public_exponent=65537,  
        key_size=2048,  
        backend=default_backend()  
    )  
    public_key = private_key.public_key()  
    return private_key, public_key
```

Penjelasan Keputusan Implementasi:

1. Eksponen 65537 dipilih karena mempercepat proses enkripsi (hanya 2 bit yang bernilai 1).
2. Key size 2048 bit dipilih karena sesuai rekomendasi NIST dan aman hingga tahun 2030.

4.1.2 Enkripsi File dan Dekripsi File dengan AES-256-GCM

Potongan Kode:

```
def generate_session_key() -> bytes:  
    """Generate random 32-byte session key for AES-256"""  
    return os.urandom(32)  
  
def encrypt_file_aes_gcm(file_data: bytes, session_key: bytes) -> dict:  
    """  
    Enkripsi file dengan AES-256-GCM  
    Returns: {  
        'ciphertext': bytes,  
        'nonce': bytes  
    }  
    """  
    nonce = os.urandom(12)  
    aesgcm = AESGCM(session_key)  
    ciphertext = aesgcm.encrypt(nonce, file_data, None)  
    return {  
        'ciphertext': ciphertext,  
        'nonce': nonce  
    }  
  
def decrypt_file_aes_gcm(ciphertext: bytes, session_key: bytes, nonce: bytes) -> bytes:  
    """Dekripsi file dengan AES-256-GCM"""  
    aesgcm = AESGCM(session_key)  
    return aesgcm.decrypt(nonce, ciphertext, None)
```

Penjelasan Keputusan Implementasi:

1. Session key 32 byte (256 bit) dipilih karena AES-256 memberikan tingkat keamanan tertinggi.

2. Nonce 12 byte dipilih karena merupakan panjang yang direkomendasikan untuk AES-GCM.
3. Nonce dari `os.urandom()` dipilih karena sumber entropi OS aman untuk kriptografi.
4. Mode GCM dipilih karena memberikan autentikasi terintegrasi, tidak seperti CBC.

4.1.3 Wrapping Session Key dengan RSA-OAEP

Potongan Kode:

```
def wrap_session_key_rsa(session_key: bytes, public_key_pem: str) -> bytes:
    """Enkripsi session key dengan RSA public key (OAEP)"""
    public_key = deserialize_public_key(public_key_pem)
    wrapped = public_key.encrypt(
        session_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return wrapped
```

Penjelasan Keputusan Implementasi:

1. Padding OAEP dipilih karena terbukti aman terhadap chosen-ciphertext attack.
2. MGF1 dengan SHA-256 dipilih agar konsisten dengan algoritma hash yang digunakan.

4.1.4 Key Derivation dengan PBKDF2

Potongan Kode:

```
def derive_key_from_password(password: str, salt: bytes, iterations: int = 100000, length: int = 32) -> bytes:
    """Derive encryption key from password using PBKDF2-HMAC-SHA256"""
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=length,
        salt=salt,
        iterations=iterations,
        backend=default_backend()
    )
    return kdf.derive(password.encode('utf-8'))
```

Penjelasan Keputusan Implementasi:

1. Iterasi 100.000 dipilih sesuai rekomendasi NIST untuk memperlambat brute force.
2. Output 32 byte dipilih karena tepat untuk kunci AES-256.
3. HMAC-SHA256 dipilih karena SHA-256 aman dan didukung luas.

4.1.5 Enkripsi Private Key dengan Password

Potongan Kode:

```
def encrypt_private_key_with_password(private_key_pem: bytes, password: str, salt: bytes = None) -> dict:
    """
    Enkripsi RSA private key dengan password menggunakan AES-256-GCM
    Returns: {
        'encrypted_key': bytes,
        'salt': bytes,
        'nonce': bytes,
        'iterations': int
    }
    """
    if salt is None:
        salt = os.urandom(16)

    iterations = 100000
    key = derive_key_from_password(password, salt, iterations)
    nonce = os.urandom(12)
    aesgcm = AESGCM(key)
    ciphertext = aesgcm.encrypt(nonce, private_key_pem, None)

    return {
        'encrypted_key': ciphertext,
        'salt': salt,
        'nonce': nonce,
        'iterations': iterations
    }
```

Penjelasan Keputusan Implementasi:

1. Private key dienkripsi dipilih agar meskipun database bocor, private key tidak dapat digunakan tanpa password user.
2. Salt 16 byte dipilih karena cukup untuk mencegah rainbow table.

4.2 Implementasi Autentikasi dengan bcrypt

Potongan Kode:

```
# Hash password untuk login (bcrypt)
hashed_password = bcrypt.hashpw(request.password.encode('utf-8'), bcrypt.gensalt())
```

```
if not bcrypt.checkpw(request.password.encode('utf-8'), user.password_hash.encode('utf-8')):
    raise HTTPException(status_code=401, detail="Invalid credentials")
```

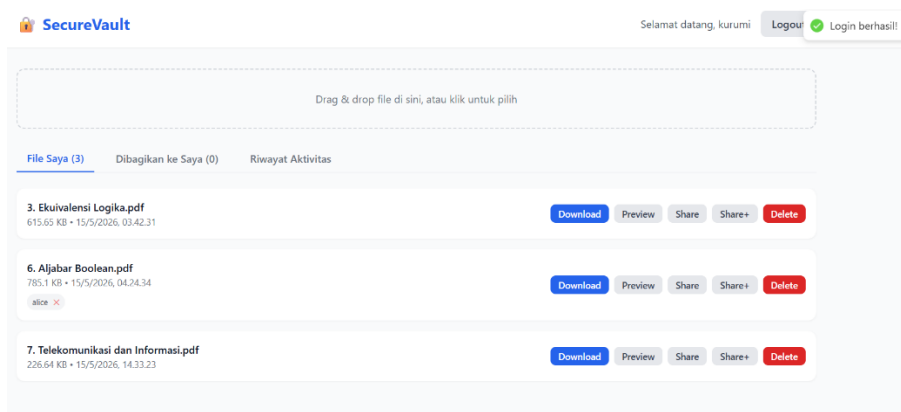
Penjelasan Keputusan Implementasi:

1. bcrypt dipilih untuk hash password karena salt sudah terintegrasi otomatis.

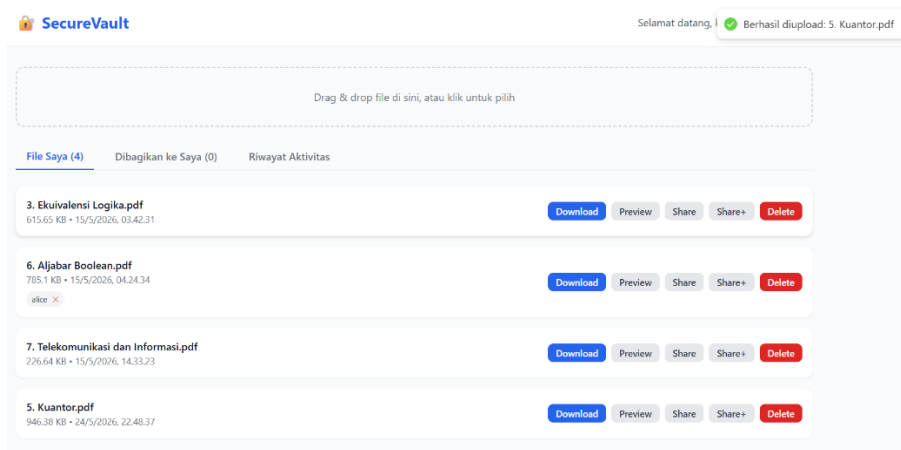
E. PENGUJIAN

5.1 Skenario Normal

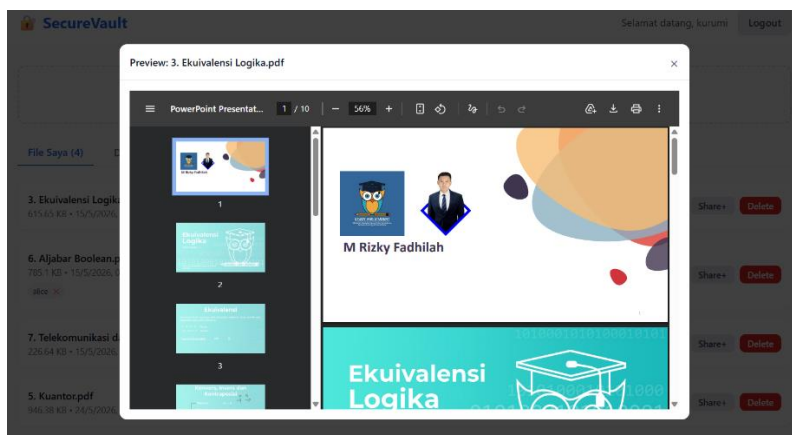
Login Berhasil:



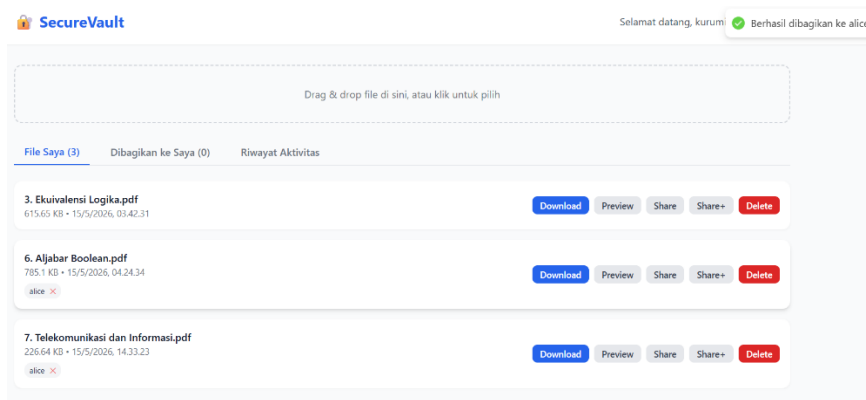
Upload file:



Preview file:

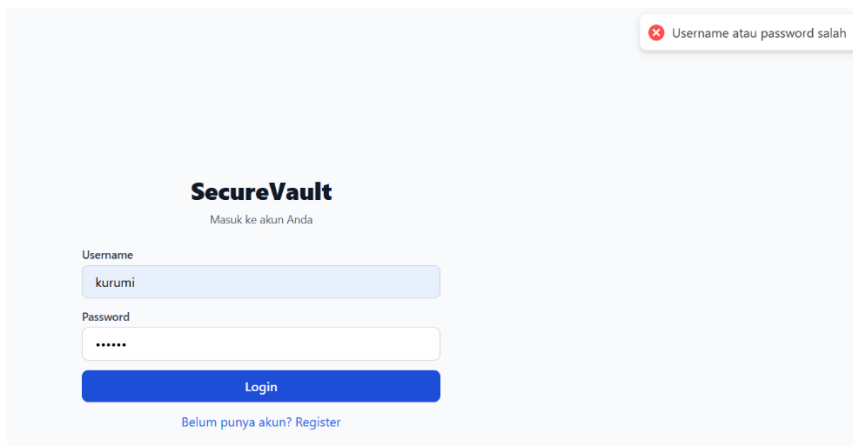


Share file:

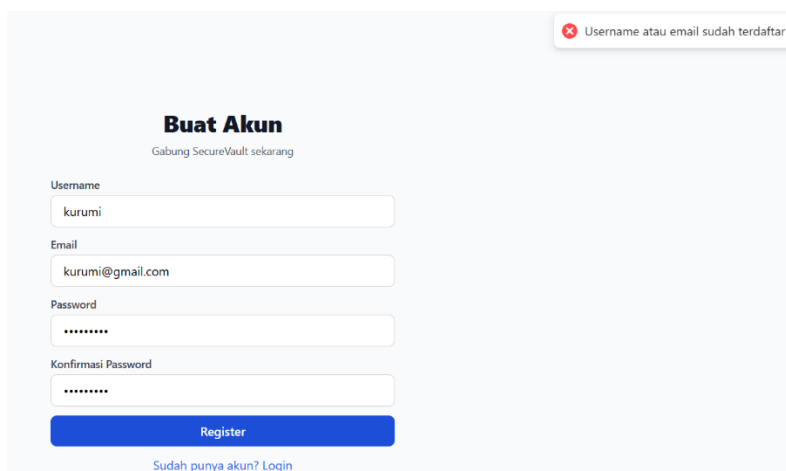


5.2 Skenario Error

Login password salah:



Registrasi username sudah ada:



F. KETERBATASAN & REFLEKSI

6.1 Apa yang Belum Bisa Dilakukan Sistem

SecureVault memiliki beberapa keterbatasan fungsional. Preview file hanya mendukung teks (.txt), gambar (.jpg, .png, .gif), dan PDF, belum bisa untuk .docx atau .xlsx. Upload file besar (>100MB) belum dioptimalkan karena seluruh file dimuat ke memory. Enkripsi client-side belum sepenuhnya diterapkan saat ini enkripsi/dekripsi terjadi di backend, sehingga server sempat melihat plaintext. Selain itu, MFA, lupa password, dan rename file juga belum tersedia.

6.2 Celah Keamanan yang Masih Ada

Terdapat beberapa celah keamanan yang perlu diperhatikan. Password dikirim melalui query parameter pada endpoint download dan share, sehingga terlihat di URL dan berpotensi terekam di log server. Tidak ada rate limiting, meskipun bcrypt dan PBKDF2 memperlambat brute force, serangan dalam skala besar tetap mungkin dilakukan. Token JWT disimpan di localStorage, rentan terhadap serangan XSS. Belum menggunakan HTTPS (masih HTTP) dan tidak ada auto-logout untuk session tidak aktif.

6.3 Rencana Perbaikan (Jika Ada Waktu Lebih)

1. Keamanan: Memindahkan password ke request body (bukan query parameter), menambahkan rate limiting (maksimal 10 percobaan login per menit), memindahkan token JWT ke httpOnly cookie, mengimplementasikan auto-logout setelah 30 menit tidak aktif, dan migrasi ke HTTPS.
2. Fitur: Memindahkan enkripsi/dekripsi ke client-side (Web Crypto API) agar server benar-benar tidak melihat plaintext, menambahkan preview untuk .docx/.xlsx, fitur lupa password, dan upload file secara chunk untuk file besar.
3. Performa: Menggunakan streaming untuk enkripsi/dekripsi file besar agar tidak membebani memory, serta menambahkan pagination pada daftar file.

6.4 Refleksi Akhir

Proyek SecureVault telah berhasil mengimplementasikan seluruh fitur wajib yang diminta. Sistem ini membuktikan bahwa hybrid cryptosystem (AES + RSA) dapat diimplementasikan dengan baik untuk penyimpanan file terenkripsi dengan fitur berbagi yang aman. Namun, seperti yang telah diuraikan, masih terdapat berbagai keterbatasan dan celah keamanan yang perlu diperbaiki.